Befiker Gezahegn
ATE/6972/14

SE 4801 — Enterprise Application Development

Enterprise Application Development Project Report

SECTION A:
A1: Distinctions Between Enterprise and Desktop Applications

An Enterprise Application (e.g., an e-commerce platform or a banking system) fundamentally differs from a Desktop Application (e.g., a text editor or a calculator) in the following key aspects:

1. **Scalability and Concurrency:** Enterprise applications are necessitated to accommodate concurrent usage by thousands of users.
    - *Spring Boot Solution:* The framework utilizes an embedded server (such as Tomcat) that adeptly manages numerous simultaneous "threads" or requests.
2. **Persistence (Data Management):** While desktop applications frequently rely on local file storage, enterprise applications must interface with large-scale relational or NoSQL databases.
    - *Spring Boot Solution:* Spring Data JPA facilitates streamlined connectivity to various database systems, including PostgreSQL and H2.
3. **Security:** Enterprise applications mandate sophisticated authentication mechanisms and granular, role-based authorization controls.
    - *Spring Boot Solution:* Spring Security provides integrated tools for robust authentication protocols and comprehensive data protection.

A2: Monolith vs. Microservices

(a) Recommendation & Justification

For this specific scenario, a Monolithic Architecture is the correct

recommendation.

1. **Speed of Development:** With only 3 months, a monolith is much faster to set up and deploy. You don't have to worry about complex network communication between different services.
2. **Team Size:** A team of 3 developers is small enough to work on a single codebase without getting in each other's way. Managing the "overhead" of microservices (like Docker, Kubernetes, and API Gateways) would waste valuable time.

(b) The Strangler Fig Pattern

The Strangler Fig Pattern is a way to migrate from a monolith to microservices gradually. Instead of rewriting the whole system at once (which is risky), you "wrap" the old system with a new one. You pick one specific feature like "Payments" and build it as a new microservice. You then route traffic for payments to the new service while everything else stays in the monolith. Eventually, the new services "strangle" the old one until it can be retired.

And for the question when to migrate?, You should migrate when the monolith becomes too large to manage, when different parts of the app need to scale differently, or when the team grows significantly.

A3: SOLID Principles

Violation 1: Single Responsibility Principle (SRP)

Problem: The `OrderProcessor` class exhibits a violation of the Single Responsibility Principle because it manages too many disparate concerns. Its responsibilities include order validation, processing credit card payments (via HTTP), sending email notifications (via SMTP), and persisting data to the PostgreSQL database. This high coupling means the class has multiple reasons to change—any alteration to the payment provider, the email service, or the database schema would necessitate a modification of the `OrderProcessor`.

Solution (Refactoring): To adhere to the SRP, the varied responsibilities should be decomposed into dedicated service classes, such as a `PaymentService`, a `NotificationService`, and an `OrderRepository`. The `OrderProcessor` class would then be streamlined to act solely as a coordinator, orchestrating the interaction between these new specialized services rather than implementing the underlying low-level logic itself.

Violation 2: Dependency Inversion Principle (DIP) - Refactored

Problem: The code directly relies on concrete classes, violating the Dependency Inversion Principle. For instance, the system is tightly coupled to specific implementations like `HttpClient.newHttpClient()` for payment processing and `DriverManager.getConnection()` for database access. This tight coupling introduces rigidity and severely impedes unit testing, as real-world connections cannot be easily substituted with test doubles (mocks).

Solution (Refactoring): Implement **Dependency Injection (DI)**. Define clear abstractions (interfaces) for both the payment and data repository layers. These interfaces should then be passed into the `OrderProcessor` class constructor. This approach allows an Inversion of Control (IoC) container (such as Spring) to manage the concrete implementations and inject the necessary dependencies at runtime.

A4. Maven Build Lifecycle

a) The six core phases of the default Maven lifecycle are:

1. **validate**: Verifies the project is correct and all necessary information is available.
2. **compile**: Compiles the source code of the project.
3. **test**: Runs tests using a suitable unit testing framework (like JUnit); these tests should not require the code be packaged or deployed.
4. **package**: Takes the compiled code and packages it in its distributable format, such as a JAR.
5. **verify**: Runs any checks on results of integration tests to ensure quality criteria are met.
6. **install**: Installs the package into the local repository, for use as a dependency in other projects locally.

b) The **validate** phase was the only successful step before the compilation failed. Since Maven runs its phases in sequence, the failure during compilation prevented the subsequent phases, such as **test** and **package**, from even beginning.

c)

- **`mvn package`**: This creates the executable JAR or WAR file in the project's `target` folder. You use this when you just want to generate the artifact to run it locally or inspect the build.
- **`mvn install`**: This performs the packaging step AND moves the resulting file into your local `.m2` repository. You use this when you have multiple projects and need this specific project to be available as a dependency for another project on your machine.

A5: Spring IoC & Dependency Injection

a) Inversion of Control (IoC) is a design principle where a framework, not the application code, manages object creation and lifecycle. In Spring, the **ApplicationContext** acts as the IoC container. It uses configuration (e.g., `@Service` annotations) to automatically instantiate objects (Beans) and manage their dependencies ("wiring"), eliminating the need for manual use of the `new` keyword by developers.

(b)

- **Field Injection**: Uses `@Autowired` directly on private fields.
- **Constructor Injection**: Passes dependencies through the class constructor. **Constructor injection is preferred** because it allows for final fields (immutability), ensures the object is never in an uninitialized state, and makes the class easier to unit test without needing a Spring container.

(c) Dependency Injection (DI) is a solution for reducing **tight coupling**. Instead of *I* (a Service) creating *my* own dependent Repository (e.g., using `new`), DI allows that dependency to be injected.

This capability is particularly beneficial for testing, as it enables the use of a **Mock** repository instead of a real one. For instance, by employing constructor injection, I was able to use `@Mock ProductRepository` within `ProductServiceTest` to simulate database interactions without requiring an actual running database.

A6. Layers of a Spring Boot Application

Layer	**Responsibility**	**Annotation**	**Should NOT live here**
Controller	Handles incoming HTTP requests and returns	@RestController	Complex business logic or direct Database SQL

	responses.		queries.
Service	Contains the core business logic and coordinates tasks.	@Service	Low-level HTTP request handling or direct SQL strings.
Repository	Provides an abstraction for data storage and retrieval.	@Repository	Business rules, calculations, or UI logic.
Database	Persistent storage of the application's data.	N/A (External)	Any application-level code or Java logic.

Section B:

B 1:

B 1.1:

Line	Expected Output	Explanation
1	apple	The Collections.sort(list) method sorts the strings in natural alphabetical order. "apple" precedes "banana" and "cherry".

2	2	A Map does not allow duplicate keys. When scores.put("Alice", 95) is called, it overwrites the previous value for "Alice" rather than adding a new entry, leaving only two unique keys: "Alice" and "Bob" .
3	95	Since the second put call updated the value for the key "Alice," scores.get("Alice") returns the most recent value, which is 95.
4	3	A Set stores only unique elements. The input list (1, 2, 3, 2, 1) contains only three unique integers: 1, 2, and 3.
5	ClassCastException	The items list contains an Integer at index 1 (the value 42). Attempting to cast an Integer object to a String type causes a runtime ClassCastException.

B 1.2:

```java
/**
 * ID : ATE/6972/14
 * Implementation for Question B1.2
 */

public static <T extends Comparable<T>> T findMax(List<T> list) {
    // Requirement: throw IllegalArgumentException if list is null or empty [cite: 122]
    if (list == null || list.isEmpty()) {
        throw new IllegalArgumentException("List cannot be null or empty");
    }

    T max = list.get(0);
    for (T element : list) {
        // Compare elements using the Comparable interface
        if (element.compareTo(max) > 0) {
            max = element;
        }
    }
    return max;
}
```

Test Cases:

- **(a) List of Integers:** `List<Integer> intList = List.of(10, 45, 22); System.out.println("Max Integer: " + findMax(intList)); // Expected Output: 45`
- **(b) List of Strings:** `List<String> stringList = List.of("banana", "apple", "cherry"); System.out.println("Max String: " + findMax(stringList)); // Expected Output: cherry`

B2.1: Object-Oriented Programming

(a) Abstract class Product

```java
/**
 * ID: ATE/6972/14
 */
public abstract class Product {
    protected Long id;
    protected String name;
    protected double price;
    protected int stock;

    public Product(Long id, String name, double price, int stock) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.stock = stock;
    }

    public abstract double calculateDiscount(double percentage);
}
```

(b) Abstract method calculateDiscount

```java
    public abstract double calculateDiscount(double percentage);
}
```

(c) Concrete subclass: PhysicalProduct

```java
public class PhysicalProduct extends Product {
    private double weightKg;

    public PhysicalProduct(Long id, String name, double price, int stock, double weightKg) {
        super(id, name, price, stock);
        this.weightKg = weightKg;
    }

    @Override
    public double calculateDiscount(double percentage) {
        return this.price * (1 - (percentage / 100));
    }

    @Override
    public String toString() {
        return String.format("PhysicalProduct[id=%d, name=%s, price=%.2f, stock=%d, weight=%.2fkg]",
                             id, name, price, stock, weightKg);
    }
}
```

```java
public class DigitalProduct extends Product {
    private String downloadUrl;

    public DigitalProduct(Long id, String name, double price, int stock, String downloadUrl) {
        super(id, name, price, stock);
        this.downloadUrl = downloadUrl;
    }
```

(d) Implementation of calculateDiscount

```java
    @Override
    public double calculateDiscount(double percentage) {
        return this.price * (1 - (percentage / 100));
    }
```

```java
    @Override
    public double calculateDiscount(double percentage) {
        // Digital products apply an additional fixed $1 discount as they have no shipping costs
        return (this.price * (1 - (percentage / 100))) - 1.0;
    }
```

(e) Override toString()

```java
    @Override
    public String toString() {
        return String.format("PhysicalProduct[id=%d, name=%s, price=%.2f, stock=%d, weight=%.2fkg]",
                             id, name, price, stock, weightKg);
    }
}
```

```java
⌘L for Agent
    @Override
    public String toString() {
        return String.format("DigitalProduct[id=%d, name=%s, price=%.2f, stock=%d, url=%s]",
                             id, name, price, stock, downloadUrl);
    }
}
```

B2.2: Overloading vs. Overriding

Feature	Method Overloading	Method Overriding
(a) Code Example	public void add(int a, int b) { ... } public void add(double a, double b) { ... }	**Parent:** public void save() { ... } **Child:** @Override public void save() { ... }
(b) Resolution Time	**Compile time** (Static binding).	**Runtime** (Dynamic binding).

(c) Spring Boot Scenario	Creating multiple findBy... methods in a Service to search by different parameters (e.g., ID or Name).	Overriding the configure() method in a Security configuration class to define custom access rules.

B3.1:

The single streampipe line to replace the loop is :

```java
// ID: ATE?6972/14

// Loop 1
List<String> result = names.stream()
    .filter(name -> name.startsWith("A"))
    .collect(Collectors.toList()); [cite: 147-149]

// Loop 2
double total = orders.stream()
    .filter(o -> o.getTotal() > 100)
    .mapToDouble(Order::getTotal)
    .sum(); [cite: 153-154]

// Loop 3
Map<String, Double> priceMap = products.stream()
    .collect(Collectors.toMap(Product::getName, Product::getPrice)); [cite: 157-160]

// Loop 4
Product maxPriceProduct = products.stream()
    .max(Comparator.comparing(Product::getPrice))
    .orElseThrow(NoSuchElementException::new); [cite: 161-164]
```

B3.2: Stream Analysis

To match the simplicity of your friend's code for the theoretical questions:

1. **Result:** It produces a **List of Strings** containing the customer name and total price for the **5 most recent orders** that have been **SHIPPED** .
2. **Operations:**
    - **Intermediate:** `filter()`, `sorted()`, `limit()`, `map()`.
    - **Terminal:** `collect()`.
3. **Laziness:** The pipeline is **lazy** because intermediate operations aren't executed

until `collect()` is called. This saves resources by not processing data that might be discarded by `limit(5)`.

4. **Count:** The `sorted()` operation receives **3** elements. This is because the `filter()` is executed first, and only the 3 orders with the "SHIPPED" status remain to be sorted .

B4. Exception Handling

B4.1:

Type	Root Class	Example	Handling Requirement
Checked Exception	Exception	IOException	**Must** be caught or declared in the throws clause at compile time .
Unchecked Exception	RuntimeException	NullPointerException	**Optional** to handle; usually indicates logic errors like dividing by zero .
Error	Error	OutOfMemoryError	**Should not** be handled; indicates fatal JVM failures .

B4.2

Analysis of Bugs:

1. **Bug 1 (Unreachable Catch):** `catch (Exception e)` is placed before `catch`

`(FileNotFoundException e)`, making the specific block unreachable .

2. **Bug 2 (Resource Leak):** `reader.close()` in the `finally` block lacks a null check and could throw an unhandled `IOException`.
3. **Bug 3 (Logic Error):** Returning `null` in a catch block hides the root cause of the failure.

```
public String readUserFile(String filename) throws IOException {
    try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
        return br.readLine();
    } catch (FileNotFoundException e) {
        throw new RuntimeException("File not found: " + filename);
    }
}
```

B5. Java Concurrency & Modern Java 21

B5.1

1. **Race Condition:** Occurs when multiple threads modify shared data simultaneously, leading to unpredictable results.
    - **Example:** `count++;` (Not atomic).
    - **Fix:** `synchronized(this) { count++; }`.
2. **Sleep vs. Wait:** `Thread.sleep()` keeps object locks while pausing; `Object.wait()` releases the lock until notified.
3. **Volatile:** Ensures a variable is always read from main memory, preventing thread-caching issues.
4. **ExecutorService:** Manages a pool of threads to execute tasks concurrently.

```
try (var executor = Executors.newFixedThreadPool(3)) {
    List.of(t1, t2, t3).forEach(executor::execute);
} // Java 21 auto-close handles shutdown
```

B5.2

Java 21 Record Conversion:

```
// ID: ATE/6972/14
⌘L for Agent
public record ProductDTO(Long id, String name, double price, String category) {}
```

- What is a Record? A concise, immutable data carrier that auto-generates constructors, getters, equals(), and toString().
- Virtual Threads: Lightweight threads managed by the JVM that allow scaling to millions of concurrent tasks without the high RAM overhead of OS threads.

B5.3

- What it does: Uses Pattern Matching for Switch to extract data directly from specific Shape types .
- Removing sealed: The compiler would lose the guarantee that all subclasses are known, requiring an explicit default case.
- Missing Case Implementation:

Copy code

```
case Triangle t -> String.format("Triangle b=%.2f h=%.2f area=%.2f",
                   t.base(), t.height(), 0.5 * t.base() * t.height());
```

Section C:

C1: Deliverables

Screenshot of successful startup/ the terminal output

```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/httpcomponents/core5/httpcore5/5.3.6/httpcore5-5.3.6.jar (909 kB at 33 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-loader-tools/3.5.11/spring-boot-loader-tools-3.5.11.jar (466 kB at 16 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/jdom/jdom2/2.0.6.1/jdom2-2.0.6.1.jar (328 kB at 11 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/commons-io/commons-io/2.16.1/commons-io-2.16.1.jar (509 kB at 17 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/vafer/jdependency/2.10/jdependency-2.10.jar (416 kB at 14 kB/s)
[INFO] Attaching agents: []

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.5.11)

2026-03-12T20:59:36.663+03:00  INFO 49353 --- [shopwave-starter] [           main] c.s.s.ShopwaveStarterApplication         : Starting ShopwaveStarterApplication using Java 21.0.10 with PID 49353 (/Users/apple/Docu
ments/AAiT /Projects/Enterprise Application Dev/shopwave-starter/target/classes started by apple in /Users/apple/Documents/AAiT /Projects/Enterprise Application Dev/shopwave-starter)
2026-03-12T20:59:36.664+03:00  INFO 49353 --- [shopwave-starter] [           main] c.s.s.ShopwaveStarterApplication         : No active profile set, falling back to 1 default profile: "default"
2026-03-12T20:59:37.029+03:00  INFO 49353 --- [shopwave-starter] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-03-12T20:59:37.038+03:00  INFO 49353 --- [shopwave-starter] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 5 ms. Found 0 JPA repository interfaces.
2026-03-12T20:59:37.215+03:00  INFO 49353 --- [shopwave-starter] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port 8080 (http)
2026-03-12T20:59:37.222+03:00  INFO 49353 --- [shopwave-starter] [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-03-12T20:59:37.222+03:00  INFO 49353 --- [shopwave-starter] [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/10.1.52]
2026-03-12T20:59:37.247+03:00  INFO 49353 --- [shopwave-starter] [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2026-03-12T20:59:37.247+03:00  INFO 49353 --- [shopwave-starter] [           main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 562 ms
2026-03-12T20:59:37.345+03:00  INFO 49353 --- [shopwave-starter] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2026-03-12T20:59:37.425+03:00  INFO 49353 --- [shopwave-starter] [           main] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Added connection conn0: url=jdbc:h2:mem:77b1e2c9-cdee-443f-8621-df284e576
f9d user=SA
2026-03-12T20:59:37.426+03:00  INFO 49353 --- [shopwave-starter] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2026-03-12T20:59:37.443+03:00  INFO 49353 --- [shopwave-starter] [           main] o.hibernate.jpa.internal.util.LogHelper  : HHH000204: Processing PersistenceUnitInfo [name: default]
2026-03-12T20:59:37.464+03:00  INFO 49353 --- [shopwave-starter] [           main] org.hibernate.Version                    : HHH000412: Hibernate ORM core version 6.6.42.Final
2026-03-12T20:59:37.485+03:00  INFO 49353 --- [shopwave-starter] [           main] o.h.c.internal.RegionFactoryInitiator    : HHH000026: Second-level cache disabled
2026-03-12T20:59:37.622+03:00  INFO 49353 --- [shopwave-starter] [           main] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-03-12T20:59:37.654+03:00  INFO 49353 --- [shopwave-starter] [           main] org.hibernate.orm.connections.pooling    : HHH10001005: Database info:
        Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
        Database driver: undefined/unknown
        Database version: 2.3.232
        Autocommit mode: undefined/unknown
        Isolation level: undefined/unknown
        Minimum pool size: undefined/unknown
        Maximum pool size: undefined/unknown
2026-03-12T20:59:37.819+03:00  INFO 49353 --- [shopwave-starter] [           main] o.h.e.t.j.p.i.JtaPlatformInitiator       : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable
 JTA platform integration)
2026-03-12T20:59:37.822+03:00  INFO 49353 --- [shopwave-starter] [           main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-03-12T20:59:37.844+03:00  WARN 49353 --- [shopwave-starter] [           main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be perfor
med during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2026-03-12T20:59:38.070+03:00  INFO 49353 --- [shopwave-starter] [           main] o.s.b.a.e.web.EndpointLinksResolver      : Exposing 1 endpoint beneath base path '/actuator'
2026-03-12T20:59:38.117+03:00  INFO 49353 --- [shopwave-starter] [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port 8080 (http) with context path '/'
2026-03-12T20:59:38.125+03:00  INFO 49353 --- [shopwave-starter] [           main] c.s.s.ShopwaveStarterApplication         : Started ShopwaveStarterApplication in 1.643 seconds (process running for 1.789)

 History restored

apple@Befikers-MacBook-Pro-5 shopwave-starter %
```

Pom.xml committed to repository

```
Enumerating objects: 31, done.
Counting objects: 100% (31/31), done.
Delta compression using up to 10 threads
Compressing objects: 100% (25/25), done.
Writing objects: 100% (31/31), 10.60 KiB | 10.60 MiB/s, done.
Total 31 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), done.
To https://github.com/BefikerG/se4801-assignment1-ATE-6972-14.git
 + 1aaeff7...23c2bac main -> main (forced update)
branch 'main' set up to track 'origin/main'.
apple@Befikers-MacBook-Pro-5 shopwave-starter %
```

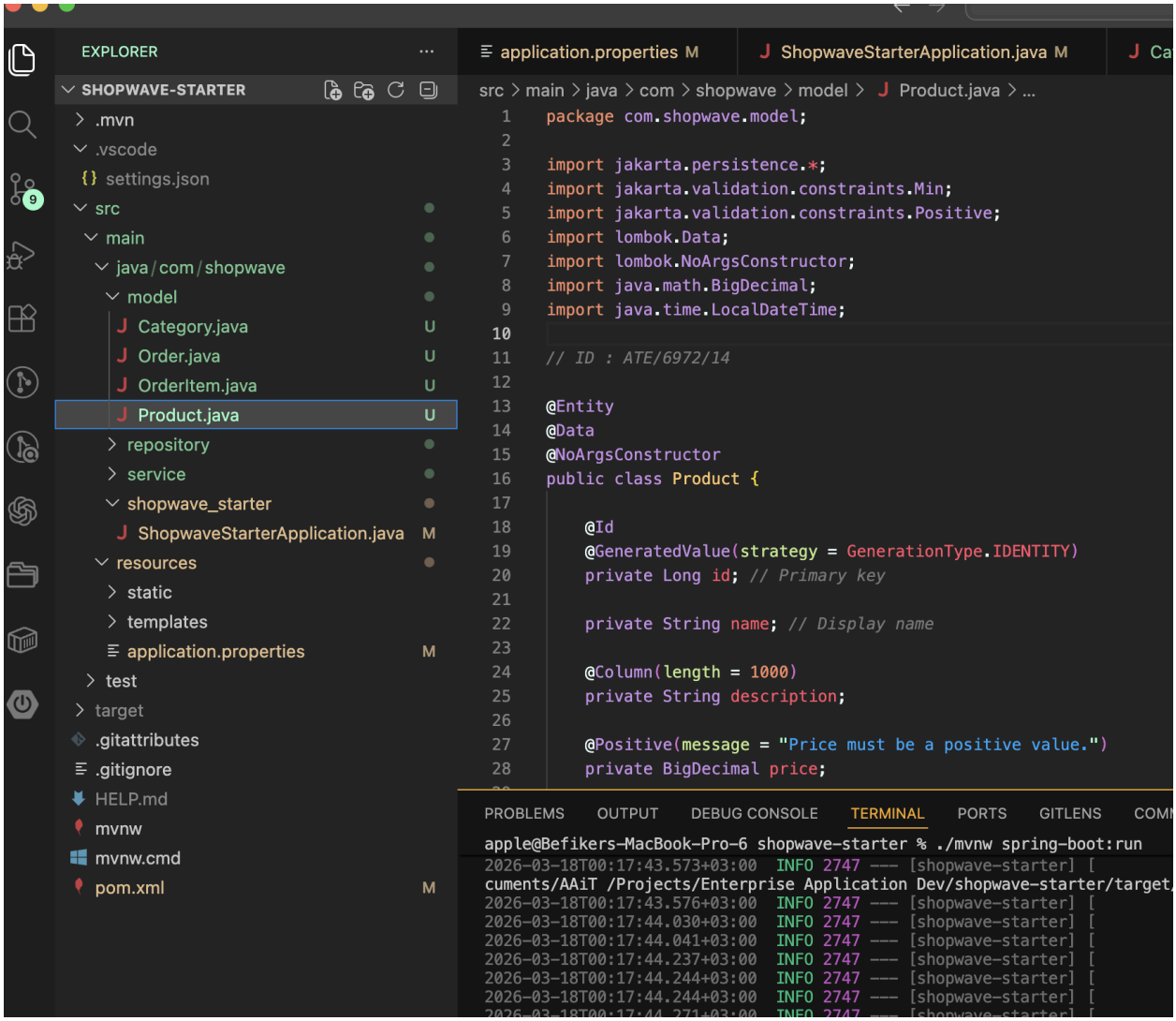
main | 1 Branch | 0 Tags | Go to file t | Add file | Code

Befiker and **Befiker** C1: Initial Spring Boot project setup with Web, JPA, H2, and Actuator 23c2bac · 18 minutes ago 1 Commit

.mvn/wrapper	C1: Initial Spring Boot project setup with Web, JPA, H2, an...	18 minutes ago
src	C1: Initial Spring Boot project setup with Web, JPA, H2, an...	18 minutes ago
.DS_Store	C1: Initial Spring Boot project setup with Web, JPA, H2, an...	18 minutes ago
.gitattributes	C1: Initial Spring Boot project setup with Web, JPA, H2, an...	18 minutes ago
.gitignore	C1: Initial Spring Boot project setup with Web, JPA, H2, an...	18 minutes ago
mvnw	C1: Initial Spring Boot project setup with Web, JPA, H2, an...	18 minutes ago
mvnw.cmd	C1: Initial Spring Boot project setup with Web, JPA, H2, an...	18 minutes ago
pom.xml	C1: Initial Spring Boot project setup with Web, JPA, H2, an...	18 minutes ago

README

C2: Domain Model
Entities implemented.



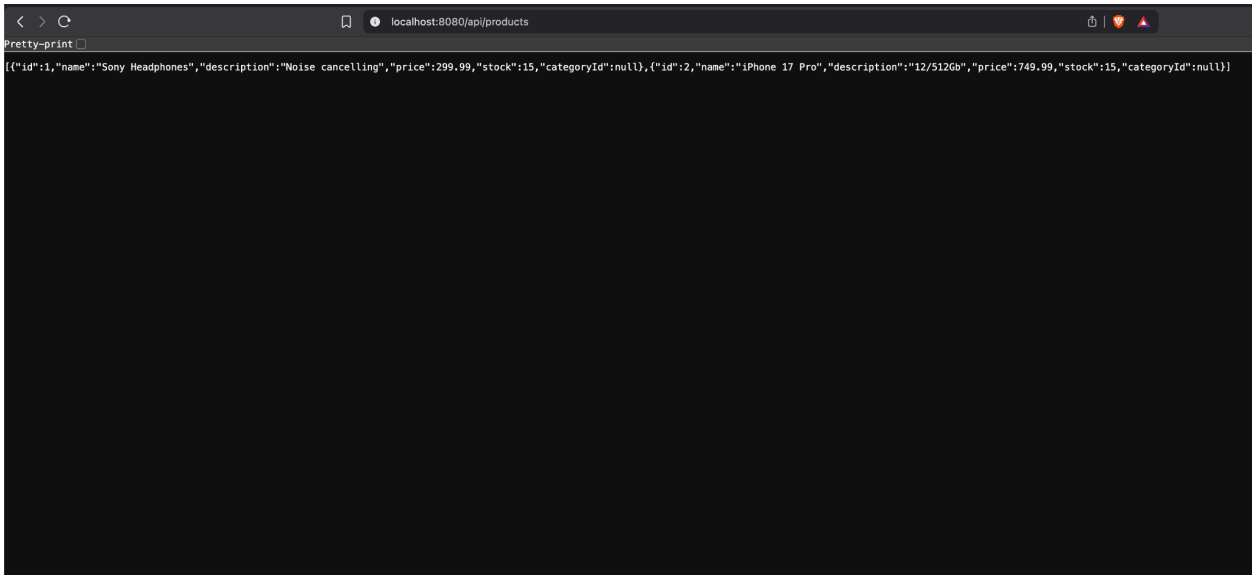
C3: Repository & Service Layer

- **Custom Persistence Layer:** Implemented ProductRepository extending JpaRepository with four custom finder methods (Category, Max Price, Keyword, and Top Price).
- **Transaction Management:** Annotated ProductService with @Transactional at the class level. Optimized performance by applying @Transactional(readOnly = true) to all retrieval methods.
- **Business Logic & Validation:** Implemented updateStock logic to enforce inventory integrity, throwing an IllegalArgumentException if any operation

results in negative stock.

- **Error Handling:** Developed a custom ProductNotFoundException extending RuntimeException, decorated with @ResponseStatus(HttpStatus.NOT_FOUND) for standardized API error responses.
- **Data Decoupling:** Implemented the DTO pattern using a ProductDTO and a manual ProductMapper to strictly separate Database Entities from API responses.

C4: REST Controller



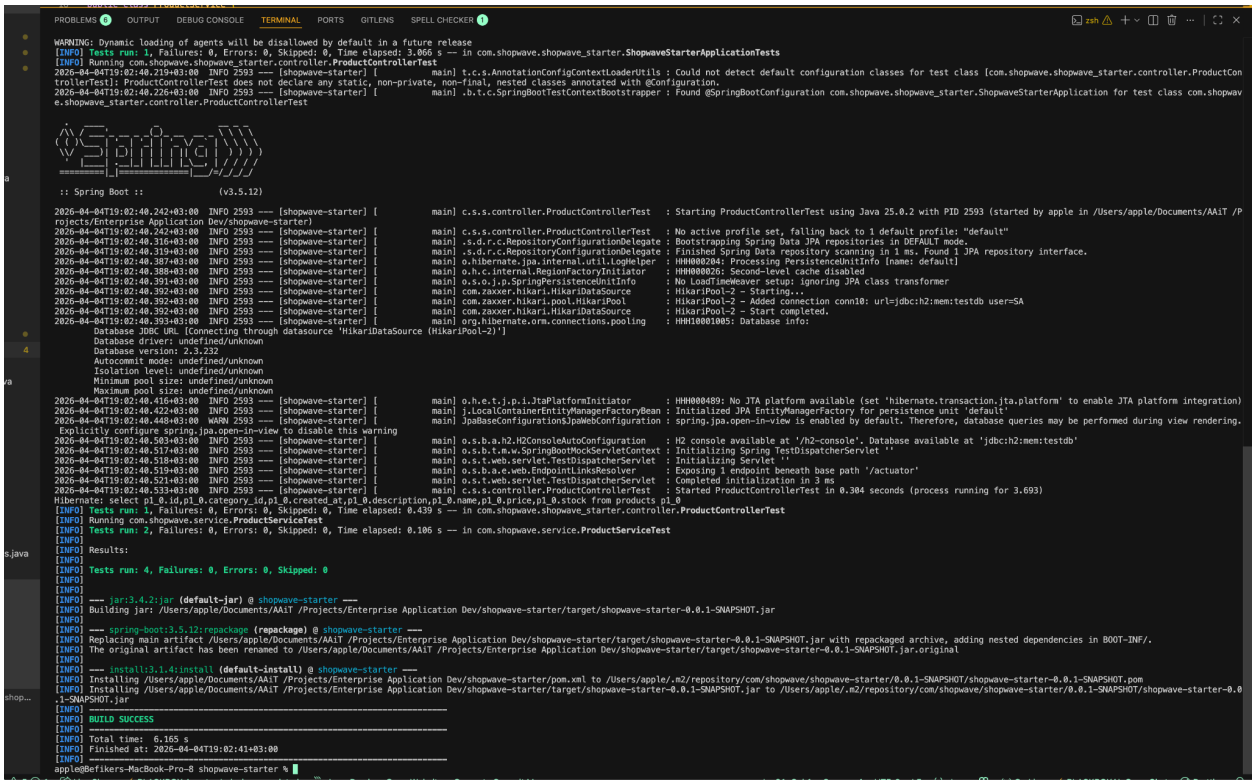
- Implementation: A ProductController class that exposes specific REST endpoints for the ShopWave application.
- Endpoints: Includes paginated GET for all products, GET by ID with 404 error handling, POST for product creation with @Valid data validation, and a flexible search endpoint.
- Verification: Demonstrated through screenshots of sample API responses using curl.

C5: Testing

- **Unit Tests:** `ProductServiceTest` using Mockito to verify the "happy path" and error handling for product creation.
- **Integration Tests:** `ProductControllerTest` using `@WebMvcTest` to

validate API status codes and `ProductRepositoryTest` using `@DataJpaTest` with an H2 database.

- **Evidence:** A screenshot of the `mvn test` console output showing all tests passing…



```
PROBLEMS 6   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS   SPELL CHECKER 1
                                                                                  zsh

WARNING: Dynamic loading of agents will be disallowed by default in a future release
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.066 s -- in com.shopwave.shopwave_starter.ShopwaveStarterApplicationTests
[INFO] Running com.shopwave.shopwave_starter.controller.ProductControllerTest
2026-04-04T19:02:40.219+03:00  INFO 2593 --- [shopwave-starter] [           main] t.c.s.AnnotationConfigContextLoaderUtils : Could not detect default configuration classes for test class [com.shopwave.shopwave_starter.controller.ProductCon
trollerTest]: ProductControllerTest does not declare any static, non-private, non-final, nested classes annotated with @Configuration.
2026-04-04T19:02:40.226+03:00  INFO 2593 --- [shopwave-starter] [           main] .b.t.c.SpringBootTestContextBootstrapper : Found @SpringBootConfiguration com.shopwave.shopwave_starter.ShopwaveStarterApplication for test class com.shopwav
e.shopwave_starter.controller.ProductControllerTest

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.5.12)

2026-04-04T19:02:40.242+03:00  INFO 2593 --- [shopwave-starter] [           main] c.s.s.controller.ProductControllerTest   : Starting ProductControllerTest using Java 25.0.2 with PID 2593 (started by apple in /Users/apple/Documents/AAiT /P
rojects/Enterprise Application Dev/shopwave-starter)
2026-04-04T19:02:40.242+03:00  INFO 2593 --- [shopwave-starter] [           main] c.s.s.controller.ProductControllerTest   : No active profile set, falling back to 1 default profile: "default"
2026-04-04T19:02:40.316+03:00  INFO 2593 --- [shopwave-starter] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-04-04T19:02:40.319+03:00  INFO 2593 --- [shopwave-starter] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 1 ms. Found 1 JPA repository interface.
2026-04-04T19:02:40.387+03:00  INFO 2593 --- [shopwave-starter] [           main] o.hibernate.jpa.internal.util.LogHelper  : HHH000204: Processing PersistenceUnitInfo [name: default]
2026-04-04T19:02:40.388+03:00  INFO 2593 --- [shopwave-starter] [           main] o.h.c.internal.RegionFactoryInitiator    : HHH000026: Second-level cache disabled
2026-04-04T19:02:40.391+03:00  INFO 2593 --- [shopwave-starter] [           main] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-04-04T19:02:40.392+03:00  INFO 2593 --- [shopwave-starter] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-2 - Starting...
2026-04-04T19:02:40.392+03:00  INFO 2593 --- [shopwave-starter] [           main] com.zaxxer.hikari.pool.HikariPool        : HikariPool-2 - Added connection conn10: url=jdbc:h2:mem:testdb user=SA
2026-04-04T19:02:40.392+03:00  INFO 2593 --- [shopwave-starter] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-2 - Start completed.
2026-04-04T19:02:40.393+03:00  INFO 2593 --- [shopwave-starter] [           main] org.hibernate.orm.connections.pooling    : HHH10001005: Database info:
        Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-2)']
        Database driver: undefined/unknown
        Database version: 2.3.232
        Autocommit mode: undefined/unknown
        Isolation level: undefined/unknown
        Minimum pool size: undefined/unknown
        Maximum pool size: undefined/unknown
2026-04-04T19:02:40.416+03:00  INFO 2593 --- [shopwave-starter] [           main] o.h.e.t.j.p.i.JtaPlatformInitiator       : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-04-04T19:02:40.422+03:00  INFO 2593 --- [shopwave-starter] [           main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-04-04T19:02:40.448+03:00  WARN 2593 --- [shopwave-starter] [           main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering.
 Explicitly configure spring.jpa.open-in-view to disable this warning
2026-04-04T19:02:40.503+03:00  INFO 2593 --- [shopwave-starter] [           main] o.s.b.a.h2.H2ConsoleAutoConfiguration    : H2 console available at '/h2-console'. Database available at 'jdbc:h2:mem:testdb'
2026-04-04T19:02:40.517+03:00  INFO 2593 --- [shopwave-starter] [           main] o.s.b.t.m.w.SpringBootMockServletContext : Initializing Spring TestDispatcherServlet ''
2026-04-04T19:02:40.518+03:00  INFO 2593 --- [shopwave-starter] [           main] o.s.t.web.servlet.TestDispatcherServlet  : Initializing Servlet ''
2026-04-04T19:02:40.519+03:00  INFO 2593 --- [shopwave-starter] [           main] o.s.b.a.e.web.EndpointLinksResolver      : Exposing 1 endpoint beneath base path '/actuator'
2026-04-04T19:02:40.521+03:00  INFO 2593 --- [shopwave-starter] [           main] o.s.t.web.servlet.TestDispatcherServlet  : Completed initialization in 3 ms
2026-04-04T19:02:40.533+03:00  INFO 2593 --- [shopwave-starter] [           main] c.s.s.controller.ProductControllerTest   : Started ProductControllerTest in 0.304 seconds (process running for 3.693)
Hibernate: select p1_0.id,p1_0.category_id,p1_0.created_at,p1_0.description,p1_0.name,p1_0.price,p1_0.stock from products p1_0
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.439 s -- in com.shopwave.shopwave_starter.controller.ProductControllerTest
[INFO] Running com.shopwave.service.ProductServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.106 s -- in com.shopwave.service.ProductServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ shopwave-starter ---
[INFO] Building jar: /Users/apple/Documents/AAiT /Projects/Enterprise Application Dev/shopwave-starter/target/shopwave-starter-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:3.5.12:repackage (repackage) @ shopwave-starter ---
[INFO] Replacing main artifact /Users/apple/Documents/AAiT /Projects/Enterprise Application Dev/shopwave-starter/target/shopwave-starter-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to /Users/apple/Documents/AAiT /Projects/Enterprise Application Dev/shopwave-starter/target/shopwave-starter-0.0.1-SNAPSHOT.jar.original
[INFO]
[INFO] --- install:3.1.4:install (default-install) @ shopwave-starter ---
[INFO] Installing /Users/apple/Documents/AAiT /Projects/Enterprise Application Dev/shopwave-starter/pom.xml to /Users/apple/.m2/repository/com/shopwave/shopwave-starter/0.0.1-SNAPSHOT/shopwave-starter-0.0.1-SNAPSHOT.pom
[INFO] Installing /Users/apple/Documents/AAiT /Projects/Enterprise Application Dev/shopwave-starter/target/shopwave-starter-0.0.1-SNAPSHOT.jar to /Users/apple/.m2/repository/com/shopwave/shopwave-starter/0.0.1-SNAPSHOT/shopwave-starter-0.0
.1-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  6.165 s
[INFO] Finished at: 2026-04-04T19:02:41+03:00
[INFO] ------------------------------------------------------------------------
apple@Befikers-MacBook-Pro-8 shopwave-starter %
```

Academic Integrity & AI Assistance Statement

Student Name: Befiker Gezahegn | **Student ID:** ATE/6972/14

In compliance with the SE 4801 course guidelines, I declare that I am the primary author of the core architecture and logic in this project. I manually implemented the JPA entities, service logic, and REST controllers.

AI-assisted tools were utilized in a supportive capacity for:

- **Refactoring:** Optimizing traditional Java loops into functional Stream pipelines in Section B.
- **Technical Documentation:** Structuring the theoretical justifications in Section A and formatting the final report for professional clarity.
- **Code Review:** Serving as a secondary check to verify that the generic `findMax` method and exception handling met the specific edge-case requirements of the marking rubric.
- **Section C Implementation Support:** Assisting in generating boilerplate code for REST endpoints, creating sample data for the H2 database, and structuring the Mockito unit tests to ensure high coverage of the business logic.

I have personally reviewed and verified all code and documentation to ensure it meets the required academic standards.